

FORMAN CHRISTIAN COLLEGE

(A Chartered University)

Course Title: Introduction to AI

Course Code: COMP360 | Section: B | Session: Spring 2026

FINAL REPORT

AI-Based Emergency Triage and Resource Allocation System

Team Name: TriageTech

Team Members

Sufyan Abbasi (271045575)

Laiba Nouman (27-1046964)

Table of Contents

1. Title and Team Information	3
2. Project Overview and Selected Vertical.....	3
3. Problem Formulation	3
3.1 Stage 1 - Classification	3
3.2 Stage 2 - Search / Optimisation	3
4. Motivation and AI Relevance	4
4.1 How AI tools were used in building this project	4
5. Related Work / Existing Approaches.....	4
6. Dataset Description and Data Understanding	5
7. Proposed AI Technique and Justification	6
7.1 Classification technique	6
7.2 Allocation technique	6
8. System Workflow / Architecture	7
9. Implementation Details	8
9.1 Components.....	8
9.2 Engineered features.....	9
9.3 Model configuration	9
9.4 Allocation logic.....	9
10. Experiments and Results	9
10.1 Setup	9
10.2 Classification results.....	9
10.3 Allocation results	10
11. Discussion of Results.....	11
12. Limitations.....	11
13. Ethical and Societal Reflection.....	11
14. Future Improvements	12
References	12

1. Title and Team Information

Project title: AI-Based Emergency Triage and Resource Allocation System.

Team name: TriageTech. This project was completed by a team of two members for COMP360 (Introduction to AI), Section B, Spring 2026, at Forman Christian College (A Chartered University).

Team Member	Roll Number	Course
Sufyan Abbasi	271045575	COMP360-B
Laiba Nouman	27-1046964	COMP360-B

2. Project Overview and Selected Vertical

Selected vertical: HealthTech.

Emergency departments work under constant pressure. Patient arrivals are unpredictable, while critical resources such as ICU beds, ventilators, and staff are limited. Two decisions must be made quickly and correctly: how urgent each patient is, and who should receive the limited resources first. Mistakes in either decision can directly affect patient survival.

TriageTech is a decision-support prototype that addresses both decisions in a single, integrated pipeline. First, a supervised Decision Tree classifies each patient into one of three urgency levels - Critical, Medium, or Low - using vital signs and presenting symptoms. Second, a cost-based search procedure (a priority queue inspired by Uniform-Cost Search) allocates the available resources so that the most urgent and highest-risk patients are served first, subject to the constraints of how many beds, ventilators, and doctors are free.

The system is delivered as a web application built with Streamlit. The interface deliberately uses plain, everyday English instead of heavy medical jargon, so that nurses and front-line staff can operate it without specialist training. Throughout, the system is positioned as a help for trained staff, not a replacement for their judgement.

3. Problem Formulation

The problem is formulated as a hybrid AI problem with two connected stages.

3.1 Stage 1 - Classification

Given a patient described by a feature vector x (heart rate, blood pressure, respiratory rate, body temperature, oxygen saturation, age, sex, arrival mode, injury, alertness, pain, and symptom indicators), predict an urgency class y in {Critical, Medium, Low}. The objective is to maximise classification quality, with particular attention to correctly identifying Critical patients, because missing a critical case is the most harmful error.

3.2 Stage 2 - Search / Optimisation

Given a set of waiting patients, each with a predicted urgency and a computed risk score, and a pool of limited resources (ICU beds, ventilators, ward beds, doctors), decide the order of service and the assignment of resources. The objective is to minimise overall patient risk and waiting time while respecting the resource constraints. This is modelled as a least-cost selection problem: each patient is a node with a cost, and the system repeatedly serves the lowest-cost (most urgent and sickest) patient that can still be given the resources it needs.

4. Motivation and AI Relevance

Traditional triage scales such as ESI, CTAS, ATS, and MTS are largely static, rule-based systems. Reviews of their performance show that conventional triage sometimes under-estimates critically ill patients, which can delay life-saving care. Triage decisions depend on several interacting variables that do not act independently, and resource allocation requires cost-aware optimisation under changing constraints. A fixed set of rules cannot systematically balance all of these factors at once, which motivates an AI-based approach.

AI is relevant here in two ways. The classification stage uses supervised learning to capture non-linear interactions among medical indicators that simple threshold rules miss. The allocation stage uses a search-based method to make structured, priority-driven decisions under constraints, rather than the naive first-come-first-served ordering.

4.1 How AI tools were used in building this project

In line with academic honesty, we note how modern AI tools were used during development. AI assistants were used as a research aid and as a coding helper - for example, to speed up background research, to suggest and debug Python code, and to organise the report. They were used as a helping hand under the team's supervision and review, not as a complete replacement for our own understanding, design decisions, or learning. Every component was checked and is understood by the team.

This mirrors the design philosophy of the system itself: the deployed tool is meant to support clinicians, not replace their judgement. AI provides a fast, consistent first opinion, while the final decision always remains with the human.

5. Related Work / Existing Approaches

Conventional triage. Standard triage instruments (ESI, CTAS, ATS, MTS, SATS) assign a severity level using structured rules and are widely used to manage patient flow and reduce crowding. Systematic comparisons report that these manual systems can vary between raters and may under-triage some high-risk patients, which justifies data-driven support.

Machine-learning triage. Recent studies apply machine learning to predict triage acuity directly from patient data. Work on the Korean Triage and Acuity Scale (KTAS) has used logistic regression, random forests, and gradient boosting (XGBoost), with ensemble models reaching high discrimination (AUROC around 0.92). A large 2026 study of more than 133,000 visits found that although XGBoost was the most accurate, a Random Forest was preferred for clinical use because it offered a better balance of accuracy

and interpretability - the same trade-off we encountered. Other studies extend ML triage to paediatric patients and incorporate free-text complaints using natural language processing to improve sensitivity.

AI for resource allocation. A separate body of work optimises hospital resources. These frameworks predict demand (for example, mechanical-ventilation need or ICU length of stay) and then assign resources using optimisation methods such as linear programming and reinforcement learning, especially during surges like COVID-19.

Gap addressed. Most existing work focuses on either triage prediction or resource optimisation in isolation, and the most accurate models are often opaque. TriageTech integrates both stages in one transparent, interpretable prototype, keeps a visible safety net for human oversight, and is deliberately lightweight so it can run on modest hardware.

6. Dataset Description and Data Understanding

Source. We use the publicly available emergency triage dataset hosted on Kaggle (the "emergency-service-triage-application" dataset), which contains the Korean Triage and Acuity Scale (KTAS) records. It holds 1,267 emergency-department visits with 24 columns covering vital signs, presentation details, and triage labels.

Target variable. The expert-assigned triage level KTAS_expert ranges from 1 (most serious) to 5 (least serious). To match the project's three-class design, we map it as follows: levels 1-2 -> Critical, level 3 -> Medium, levels 4-5 -> Low.

Features used. We use only information that is genuinely available at triage time, to avoid label leakage. The main features are:

Feature	Meaning	Type
HR	Heart rate (beats/min)	Numeric
SBP / DBP	Blood pressure (top / bottom)	Numeric
RR	Breathing rate (breaths/min)	Numeric
BT	Body temperature (Celsius)	Numeric
Saturation	Oxygen level (%)	Numeric
Age, Sex	Patient age and sex	Numeric / coded
Arrival mode	How the patient arrived	Categorical
Injury	Injury / accident or illness	Categorical
Mental	Alertness (AVPU-style)	Categorical
Pain, NRS_pain	Pain present and pain score 0-10	Coded / numeric
Symptom flags	Chest pain, breathing, etc. from complaint	Binary (derived)
KTAS_expert	Expert urgency level (target)	Target

Data understanding and challenges. Several real-world issues were found and handled during preprocessing:

- The vital-sign columns are stored as text with comma decimals (for example "3,95"); they were converted to proper numbers.
- Oxygen saturation is missing for about 54% of records; missing numeric values were filled with the column median, and the pain score was set to 0 when no pain was reported.
- The classes are imbalanced - Low 534, Medium 487, Critical 246 - so the model uses balanced class weights to avoid ignoring the rarer Critical cases.
- Outcome columns that would leak the answer (the nurse's own triage KTAS_RN, mistriage, disposition, diagnosis, length of stay) were excluded from the inputs.
- The data comes from a single setting and may not represent every population equally, which we treat as an ethical limitation.

7. Proposed AI Technique and Justification

The project combines two techniques covered in the course: a Decision Tree for classification and a cost-based search (priority queue) for allocation. Both were chosen after comparing them against simpler and more complex alternatives.

7.1 Classification technique

We compared several standard classifiers using the same features and the same five-fold cross-validation (averaged over multiple random seeds). The results are shown below.

Technique	Type	Cross-val accuracy	Easy to interpret?
Decision Tree (chosen)	Single tree	68.6%	Yes - readable rules
Logistic Regression	Linear model	68.0%	Partly
k-Nearest Neighbours	Instance-based	65.2%	No
Gaussian Naive Bayes	Probabilistic	62.2%	Partly
Random Forest	Ensemble (many trees)	71.7%	No - opaque

Justification for the Decision Tree. It gives the best accuracy among the simple, interpretable models and beats Logistic Regression, k-Nearest Neighbours, and Naive Bayes. It naturally handles mixed numeric and categorical medical data, needs no feature scaling, and models non-linear interactions between indicators. Most importantly, its decisions can be read as a chain of simple yes/no questions, which is essential for a clinical decision-support tool where staff must be able to understand and trust the reasoning. The Random Forest is about three points more accurate, but it is an opaque ensemble; using it would trade away the interpretability that the project specifically values, so we keep the single Decision Tree as the main model and note the ensemble as a future option.

7.2 Allocation technique

For resource allocation we compared the following strategies:

- **First-Come-First-Served:** simple, but ignores urgency and risk, so critical patients can wait behind minor cases - unacceptable in an emergency.
- **Fixed priority sorting:** better, but a plain sort does not cleanly handle limited, competing resources or tie-breaking by risk.
- **Cost-based search / priority queue (chosen):** models each patient as a node with a cost and always serves the lowest-cost (most urgent, highest-risk) patient next, exactly like Uniform-Cost Search expanding the cheapest frontier node. It handles constraints and tie-breaking in a principled, transparent way.
- **Linear programming / reinforcement learning:** powerful for large-scale optimisation, but heavy, data-hungry, and opaque - overkill for a one-semester, interpretable prototype.

Justification for cost-based search. The priority-queue approach is optimal for our cost model, directly supports the constraints (limited beds, ventilators, doctors), is computationally light, and remains easy to explain - matching the instructor's advice to keep the implementation simple and clearly separated from the classification stage.

8. System Workflow / Architecture

The pipeline has two clearly separated stages. Stage 1 turns raw patient information into an urgency class. Stage 2 turns urgency classes into a resource plan. A safety net of red-flag vital checks sits between them and can escalate a patient the model under-rated, with the reason shown on screen.

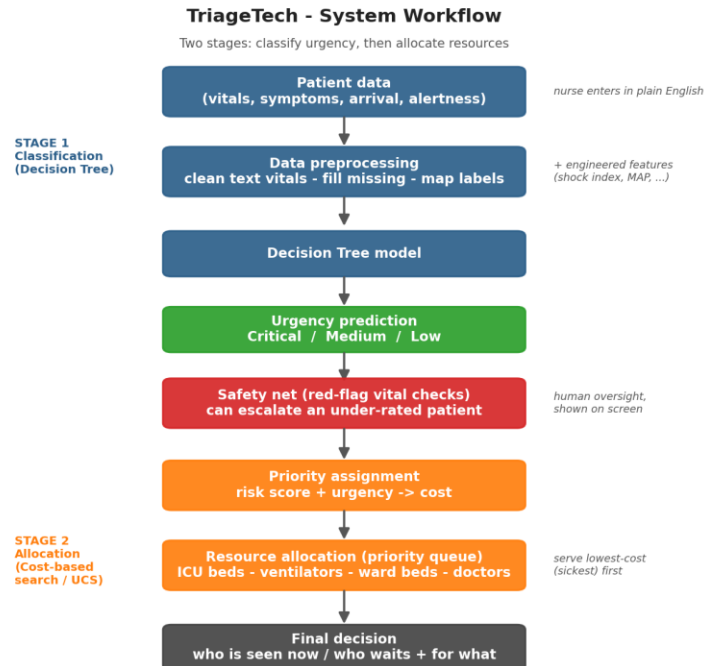


Figure 1. End-to-end system workflow: classification, safety net, and cost-based allocation.

Step by step: patient data is entered in plain English, cleaned and completed during preprocessing (including engineered clinical features), and passed to the Decision Tree, which outputs Critical / Medium / Low. The red-flag check may raise the level if dangerous vitals are present. A risk score and the urgency are combined into a cost; the priority queue then assigns ICU beds, ventilators, ward beds, and doctors in order of increasing cost until resources run out, producing the final decision of who is seen now and who waits and for what.

9. Implementation Details

The system is written in Python and organised into a small, clear set of modules, with a Streamlit front end. It is intentionally lightweight and runs comfortably on a laptop with 8 GB of RAM.

9.1 Components

- **config:** single source of truth for feature names, the KTAS-to-three-class mapping, plain-English labels, and normal vital ranges.
- **data_prep:** loads the CSV, converts text vitals to numbers, fills missing values, derives symptom flags from the complaint text, and adds engineered features.

- **model:** trains, evaluates, and saves the Decision Tree, and predicts the urgency for a single live patient.
- **allocation:** computes the risk score and red flags, performs the escalation safety net, and runs the priority-queue allocation under resource limits.
- **app (Streamlit):** five plain-English pages - Check a patient, Waiting list, Quick demo, How accurate is it, and About.

9.2 Engineered features

Because a Decision Tree splits on one value at a time, it cannot compute ratios by itself. We therefore add clinically meaningful derived features from the same readings the nurse already enters: shock index (heart rate divided by top blood pressure), pulse pressure, mean arterial pressure, and a count of how many vital signs are outside the normal range. These give the tree sharper split points and improve accuracy without asking the nurse for any extra input.

9.3 Model configuration

The final model is a scikit-learn `DecisionTreeClassifier` with `max_depth = 6`, `min_samples_leaf = 5`, cost-complexity pruning (`ccp_alpha = 0.005`), and balanced class weights. These values were selected by a cross-validation sweep to maximise stable accuracy while keeping the tree shallow and readable and preventing it from memorising the training data.

9.4 Allocation logic

Each patient receives a risk score (0-100) computed from abnormal vitals in the style of an early-warning score. The allocation cost places Critical patients ahead of Medium and Low, and within a class serves the higher-risk patient first, with arrival order as a fair tie-breaker. Resource needs are defined per level: Critical needs an ICU bed and a doctor (plus a ventilator if breathing is compromised), Medium needs a ward bed and a doctor, and Low needs a doctor only. The priority queue (a binary min-heap) repeatedly removes the lowest-cost patient and assigns resources if all required ones are free, otherwise that patient waits, and the bottleneck resource is reported.

10. Experiments and Results

10.1 Setup

The 1,267 records were split 80/20 into training (1,013) and test (254) sets with stratification. Because the raw file is ordered, model selection used five-fold cross-validation with shuffling, averaged over several random seeds for stability.

10.2 Classification results

The final Decision Tree achieves 67.3% accuracy on the held-out test set and 68.8% (plus or minus 1.1%) under cross-validation. Adding the engineered features together with hyper-parameter tuning improved the cross-validation accuracy from about 67.2% to 68.8% while keeping a single interpretable tree.

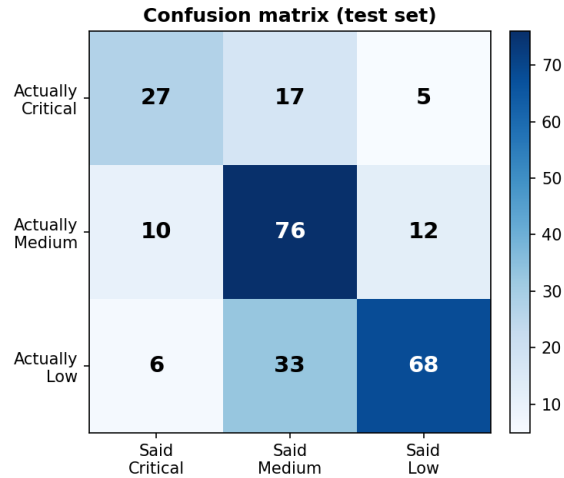


Figure 2. Confusion matrix on the test set (rows = true class, columns = predicted).

Per-class performance on the test set is shown below.

Urgency	Precision	Recall	F1-score	Patients
Critical	0.63	0.55	0.59	49
Medium	0.60	0.78	0.68	98
Low	0.80	0.64	0.71	107

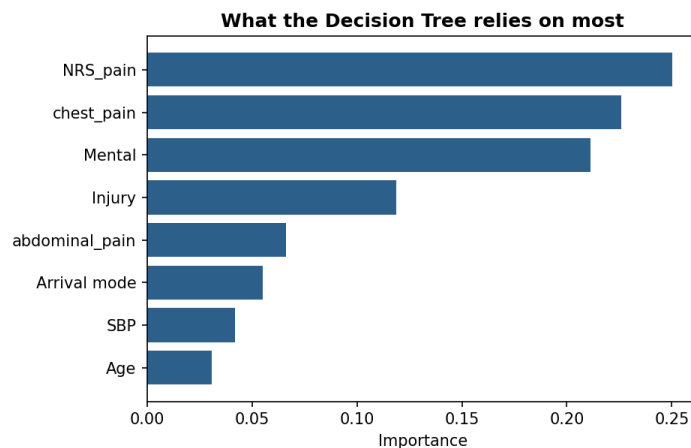


Figure 3. Features the Decision Tree relies on most.

The most influential features - pain score, chest pain, alertness, and injury - are clinically sensible drivers of urgency, which increases confidence that the model is learning meaningful patterns rather than noise.

10.3 Allocation results

In a worked example with ten waiting patients and limited resources (two ICU beds, two ventilators, five ward beds, three doctors), the priority queue served the two highest-risk Critical patients first and correctly placed a third Critical patient in a waiting state for an ICU bed once the beds were exhausted. A

lower-priority patient then had to wait for a doctor after the doctors were used up. The safety net also escalated a patient whose vitals were dangerous but whom the model had rated only Medium, and clearly displayed the reason. This demonstrates correct, constraint-aware, priority-driven behaviour.

11. Discussion of Results

The Decision Tree is competitive with, and slightly better than, the other simple models, and it trails the opaque Random Forest by only about three points. Given that the project explicitly values interpretability, this is a reasonable and defensible trade-off. The per-class results show strong performance on Medium and Low cases. The weakest point is Critical recall (0.55): the model alone misses some critical patients, which is the most dangerous type of error in triage.

This is exactly why the system includes the vital-sign safety net. Objective danger signs (for example very low oxygen, very low blood pressure, or a dangerous heart rate) automatically raise a patient's priority regardless of the model's output, and the escalation is shown to the nurse with its reason. The safety net therefore compensates for the model's main weakness while preserving transparency and human oversight.

The allocation stage behaved correctly in all tested scenarios, always serving the sickest reachable patient first and respecting the resource limits. Overall the results support the core claim of the project: combining an interpretable classifier with a transparent search procedure produces sensible, explainable triage and allocation decisions.

12. Limitations

- Accuracy is moderate (about 69%). A single Decision Tree using vitals has a real ceiling here because the expert label also depends on clinical nuance and chief-complaint detail that are not fully captured.
- Critical recall is the weakest area; the safety net reduces but does not eliminate the risk of under-triage.
- The dataset is relatively small (1,267 records) and comes from a single setting, so the model may not generalise to other populations.
- Oxygen saturation is missing for over half the records and had to be imputed, which adds uncertainty.
- The resource model is simplified - fixed needs per urgency level, no time dynamics or patient deterioration while waiting.
- Symptom flags are derived by keyword matching, which is coarse compared with full natural-language processing.
- This is an academic prototype and has not been clinically validated; it must not be used for real patient care.

13. Ethical and Societal Reflection

Fairness and bias. If historical emergency data reflects systemic inequalities, an AI trained on it can reproduce those biases. Because triage and resource decisions directly affect outcomes, fairness requires careful dataset auditing, transparency, and ongoing monitoring so that vulnerable groups are not disadvantaged.

Transparency and accountability. The system uses an interpretable Decision Tree and shows its red-flag reasoning on screen, so a human can understand and challenge each recommendation. Clear responsibility must remain with the clinical staff who make the final call.

Over-reliance and human oversight. There is a real risk that busy staff might over-trust automated output and reduce their own critical judgement. The tool is deliberately framed as decision-support, not decision-replacement, and the interface reinforces this.

Data dependence and privacy. The approach relies on accurate, timely data, which is not guaranteed in fast-paced emergencies; incomplete or delayed readings reduce reliability. Health data is also highly sensitive and must be handled with appropriate privacy safeguards.

14. Future Improvements

- Offer an optional Random Forest or gradient-boosting model for higher accuracy, paired with explainability tools (such as SHAP) to retain interpretability.
- Use natural-language processing on the chief-complaint text to extract richer symptom features and improve sensitivity, especially for Critical cases.
- Apply cost-sensitive learning or threshold tuning to raise Critical recall and further reduce under-triage.
- Train and validate on larger, more diverse, and local datasets, with explicit fairness audits across patient groups.
- Extend allocation to a time-aware model that accounts for patient deterioration while waiting, possibly using linear programming or reinforcement learning.
- Add probability calibration, logging, and drift monitoring, and explore secure integration with hospital systems and real-time vitals - followed by proper clinical validation.

References

1. Modern Triage in the Emergency Department. <https://pmc.ncbi.nlm.nih.gov/articles/PMC3021905/>
2. Medical Emergency Triage and Treatment System (METTS): A New Protocol in Emergency Medicine. <https://www.sciencedirect.com/science/article/abs/pii/S0736467908004460>
3. Triage Performance in Emergency Medicine: A Systematic Review. <https://www.sciencedirect.com/science/article/abs/pii/S0196064418312824>
4. Machine Learning-Based Prediction of Korean Triage and Acuity Scale Level in Emergency Department Patients. <https://pmc.ncbi.nlm.nih.gov/articles/PMC6859273/>
5. Interpretable Machine Learning for Emergency Department Triage Using the KTAS (133,198 patients). <https://www.mdpi.com/2075-4418/16/6/954>

6. Predicting Triage of Pediatric Patients in the Emergency Department Using Machine Learning. <https://link.springer.com/article/10.1186/s12245-025-00861-z>
7. An AI-based Multiphase Framework for Improving Mechanical Ventilation Availability in Emergency Departments. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10986051/>
8. Emergency Service Triage Dataset (KTAS), Kaggle. <https://www.kaggle.com/datasets/ilkeryildiz/emergency-service-triage-application>